UCLA Samueli School of Engineering

Digital Tapeout: Verification

Agenda

1) Verification overview
2) SVA
3) UVM
4) Advanced debugging
5) Functional & Code coverage
6) Formal verification + Logic Equivalence Checking (LEC)
7) Tools
8) SPI

What is Design Verification

VERIFICATION: WAY TO CHECK YOUR DESIGNS!

Dynamic verification:

- Give your system inputs, watch your signals change cycle-by cycle
- Dependent on quality of stimulus user injects
- UVM, emulation, etc

Static Verification:

- Analyzes underlying gate structure using math, compiler rules or boolean logic maps
- Evaluates trillions of possible states at once

SVA

Immediate Assertions vs Concurrent Assertions

Immediate Assertions:

- Execute like any other statement inside an always block
  - Must be inside proderal blocks (always, initial, task)
- Test combinational conditions (does not reference a clock)
- Syntax: label: assert (expression) action;
- Severity: $fatal, $error (default), $warning, $info
- Use case: verifying combinational states, one-hot flags

```
always_comb begin
  BUS_ERROR: assert ($onehot(bus_grant)) else $fatal("bus_grant does not contain exactly one 1");
end
```

SVA

Immediate Assertions vs Concurrent Assertions

Concurrent Assertions:

- Executes based on clocks (captures the value of the signals before the clock edge rises) - preponed, active, observed regions
- Test anything that depends on time (variables on clock edges, patterns over multiple cycles))
- Syntax: label: assert property (prop_name);
- Use case: verifying sequential stages, FSM transisitons

```
SUCCESS: assert property (@(posedge clk) ready |-> valid);
```

SVA Operators

Boolean: combinational logic checks

Sequences:

- Delays ##2, ##[m:n]
- Repetition: [*n], [->n], [=n]

Implication operators:

- Overlapping (check on same cycle as trigger): |->
- Non-overlapping(check consequence on next clock cycle): |=>
  - Functionally equivalent to |-> ##1

SVA assertion directives

Assert: validates that design is behaving correctly

Assume: constraints input, says you can assume that the statement is always true

```
VALID_MODE: assume property (@(posedge clk) mode != 2'b11);
```

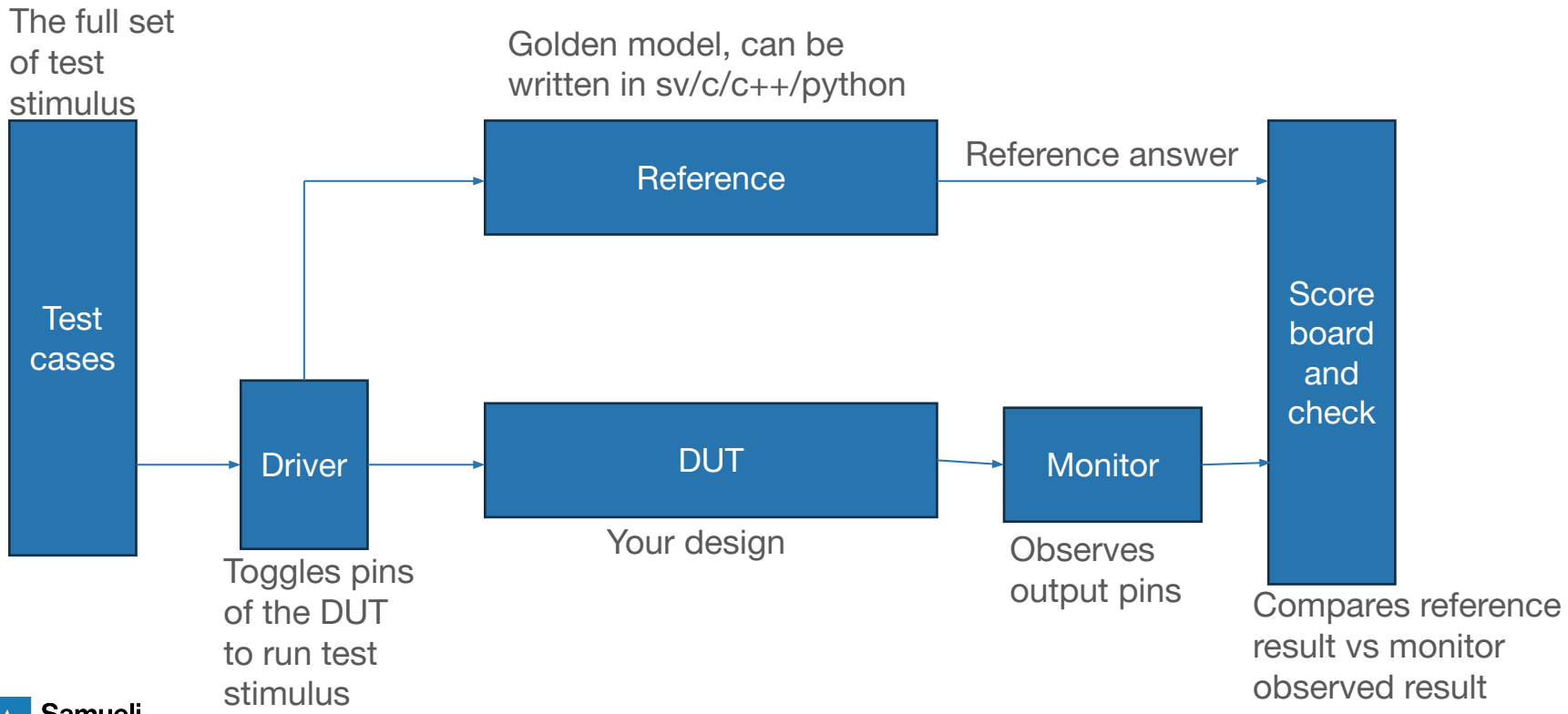
Cover: keeps score of whether specific scenarios or complex sequences actually occurred during the test

```
BURST_COV: cover property (@(posedge clk) req ##1 req);
```

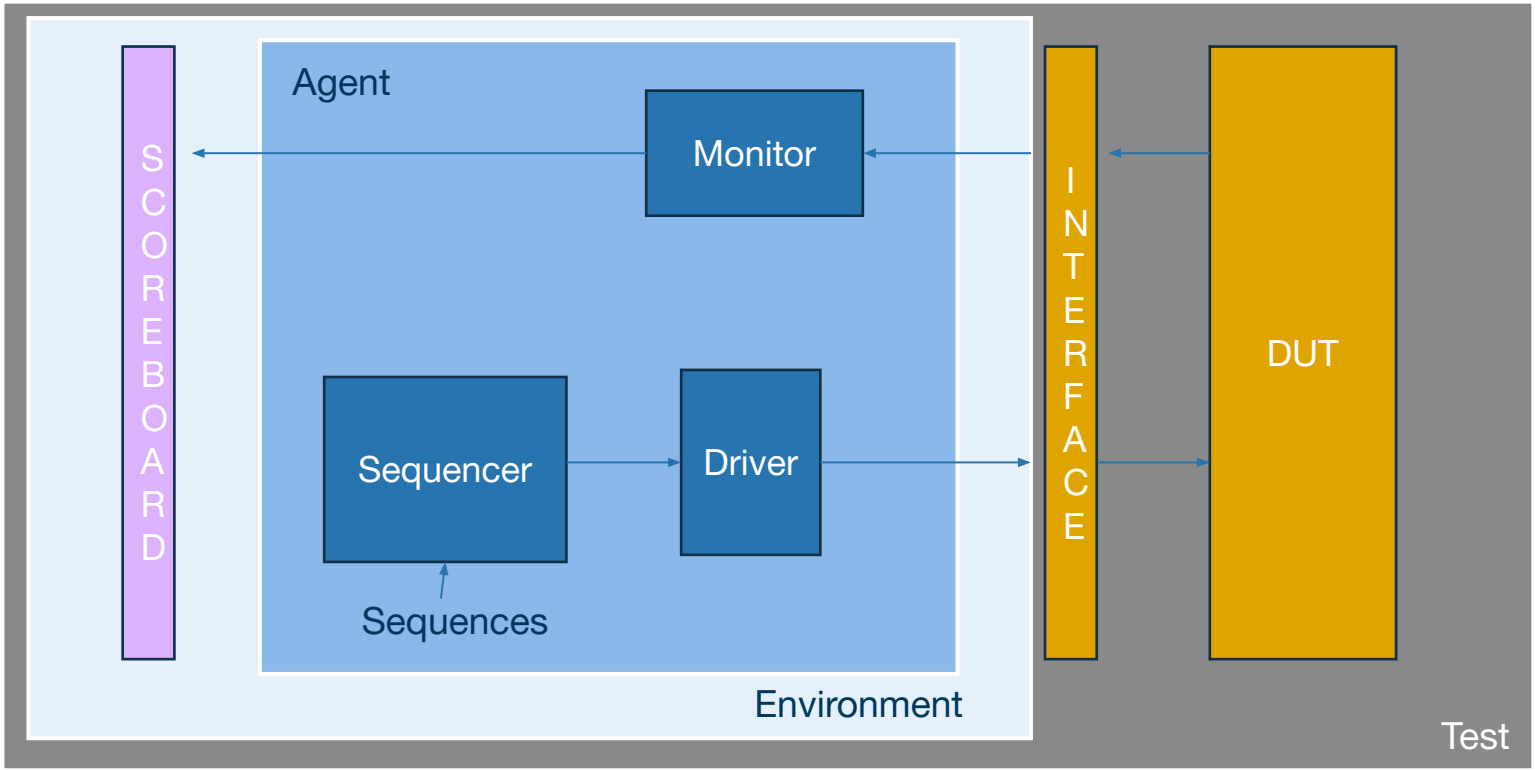
Expect: pauses execution of program until specific sequence completes successfully

```
expect (@(posedge clk) ##[1:10] bus_idle) else $error("Bus timed out!");
```

Test benching



UVM


Agent
Monitor
SCOREBOARD
INTERFACE
DUT
Sequencer
Driver
Sequences
Environment
Test

UVM Example: Two bit adder sequence

```
class adder_transaction extends uvm_sequence_item;
  `uvm_object_utils(adder_transaction) // Register with UVM factory

  rand logic [1:0] a; // Input A
  rand logic [1:0] b; // Input B
  logic [2:0] result; // Output from DUT

  function new(string name = "adder_transaction");
    super.new(name);
  endfunction
endclass
```

UVM Example: Two bit adder sequencer

```
class adder_sequence extends uvm_sequence #(adder_transaction);
  `uvm_object_utils(adder_sequence)

  task body();
    repeat(10) begin // Generate 10 random additions
      req = adder_transaction::type_id::create("req");
      start_item(req);
      if (!req.randomize()) `uvm_error("SEQ", "Randomization failed")
      finish_item(req); // Sends to Sequencer -> Driver
    end
  endtask
endclass
```

UVM Example: Two bit adder Driver

```
class adder_driver extends uvm_driver #(adder_transaction);
  `uvm_component_utils(adder_driver)
  virtual adder_if vif; // Virtual interface to the pins

  task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req); // Get item from sequencer
      @(posedge vif.clk);
      vif.a <= req.a;
      vif.b <= req.b;
      seq_item_port.item_done(); // Acknowledge completion
    end
  endtask
endclass
```

UVM Example: Two bit adder Monitor

```
class adder_monitor extends uvm_monitor;
  `uvm_component_utils(adder_monitor)
  virtual adder_if vif;
  uvm_analysis_port #(adder_transaction) item_collected_port;

  task run_phase(uvm_phase phase);
    forever begin
      adder_transaction tr = adder_transaction::type_id::create("tr");
      @(posedge vif.clk);
      tr.a = vif.a;
      tr.b = vif.b;
      tr.result = vif.result; // Capture actual output
      item_collected_port.write(tr); // Broadcast to Scoreboard
    end
  endtask
endclass
```

UVM Example: Two bit adder Scoreboard

```
class adder_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(adder_scoreboard)
  `uvm_analysis_imp_decl(_mon)
  uvm_analysis_imp_mon #(adder_transaction, adder_scoreboard) mon_imp;

  // The "Reference Model" is inside this check function
  virtual function void write_mon(adder_transaction tr);
    logic [2:0] expected = tr.a + tr.b; // High-level behavioral logic

    if (tr.result === expected)
      `uvm_info("PASS", $sformatf("%0d + %0d = %0d", tr.a, tr.b, tr.result), UVM_LOW)
    else
      `uvm_error("FAIL", $sformatf("Got %0d, Expected %0d", tr.result, expected))
  endfunction
endclass
```

Phases

Phase: standardized period of time where a specific subset of tasks should be performed by all components in the testbench

Common phases:

Build_phase: configure and create components

Connect_phase: wiring ports between components

End_of_elaboration phase: final check of setup

Start_of_simulation_phase: print initial sim messages

Run_phase: where simulation runs and simulation time passes

Extract_phase/final_phase/report_phase: final stats and pass/fail messages printed, testbench is cleaned up

Connecting it all together

Connecting hardware to software

UVM Config DB: lets UVM components access physical pins, accessed by driver

Connecting components to each other

Phases: When components are active relative to each other

TLM: The wiring of the different UVM components, lets user pass transactions between components

Connecting variations or updates:

UVM factory: creates variations of an instantiation and allows substitutions of components

Advanced debugging

How does a simulator work?

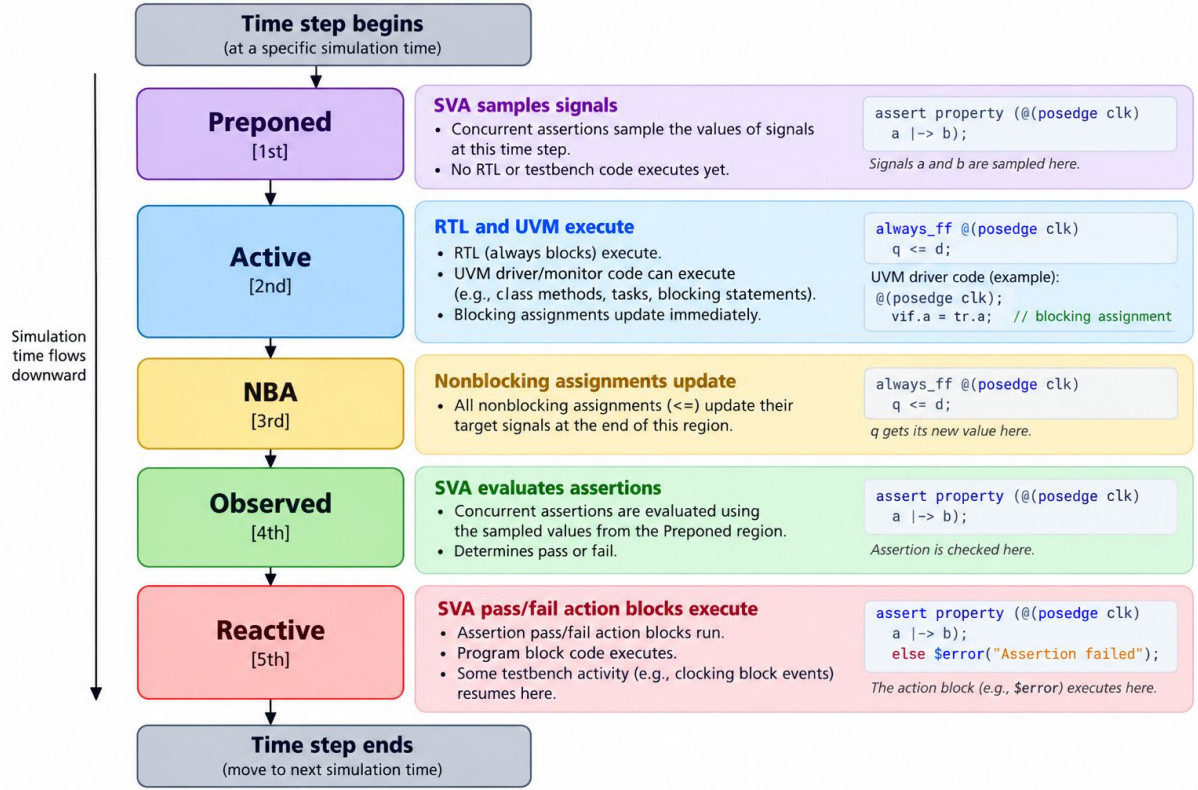
SV = parallel, testbench = sequential

Delta cycles: split up each time step into infinitesimally small steps of frozen time

Any problems with this?

Race condition: different results based on whether RTL executes first or testbench executes first

Simulator Timing



Advanced Debugging

Clocking blocks

```
clocking tb_cb @(posedge clk);
  default input #1step output #0; // #1step samples right before clock edge


  output req;
  input  ack;
  output data;
endclocking
```

Advanced debugging

X-propogation

X: unknown or unitinitialized value

- Uninitialized memory/registers
- setup/hold violations
- Bus contention (trying to drive a wire with 1 and 0 at the same time)
- Out of bounds array indexing

X-optimism

Simulator treats X like a logical 0 or 1 in conditional logic

X-pessimism

When simulator encounters X, it forces everything it touches to become X

What does large mean?

STATE SPACE: SET OF ALL POSSIBLE STATES A DESIGN CAN BE IN

Consider:

```
reg q;
```

- q can be either 0 or 1 □ state space size = 2 states

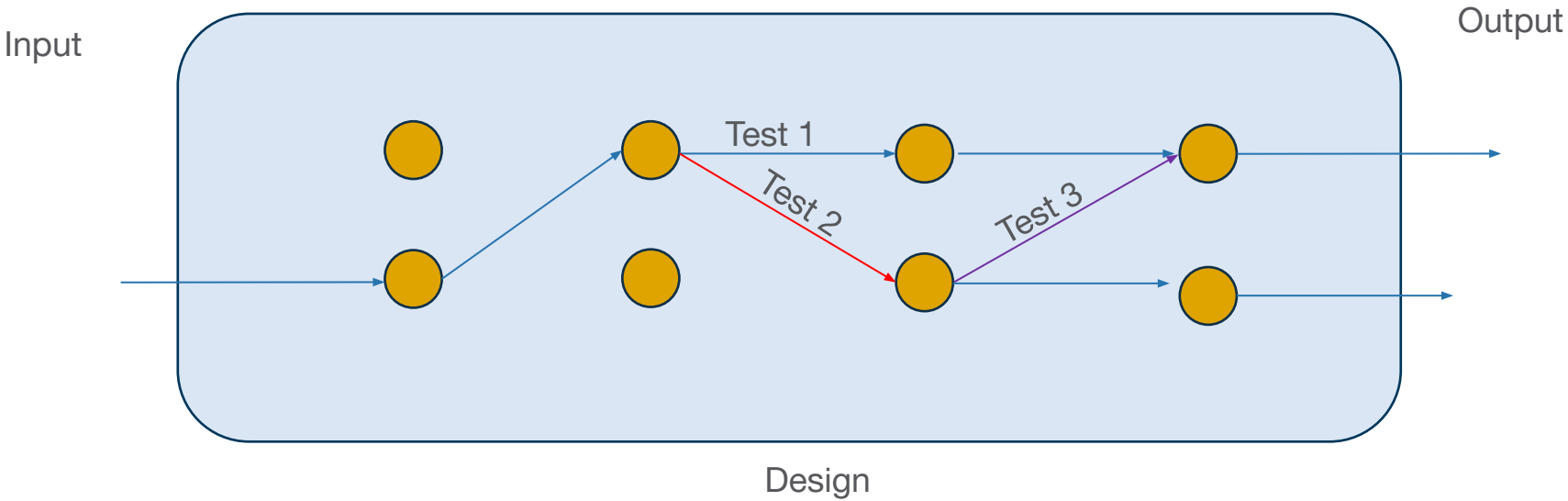
How many states are there for N flip flops? 2^N

How many states are there for this code?

```
wire [1:0] a, b;
wire [1:0] result;
assign result = a & b;
```

1 state – combinational circuits have no memory

Functional Verification and State Space



When are we done testing a design?

Code Coverage: how much of the code did we actually test?

Metrics:

Line/statement: Did every line of code get executed

Branch: for all if/else and cases, did we try every possible true and false path?

Toggle: Did every bit of every net change from 0 -> 1 and 1 -> 0

States (for FSM): Did we visit every state and test all legal transitions in the FSM?

100% code coverage does NOT mean design is bug free

Functional coverage

Coverpoints: all cases we want to cover for a specific reg

ex: For an ALU, we want to test all possible operations (add, sub, shift, xor)

- Can use to define corner cases as well:
  ex: testing math with zeros, maximum positive and negative values
- Keep track of cross coverage:
  ex: adding with maximum positive and negative values

Covergroups: groups together coverpoints (somewhat like a class of coverpoints)

Coverage: implementation example

```
covergroup alu_coverage_cg with function sample(logic [3:0] alu_ctrl, logic [31:0] a, logic [31:0] b);

cp_op: coverpoint alu_ctrl {
    bins ops[] = {4'b0000, 4'b0001, 4'b0010, 4'b0110, 4'b0111};
    illegal_bins others = default; // Flags an error if an undefined op appears
  }
cp_a_values: coverpoint a {
    bins zeros    = {32'h0};
    bins max_pos  = {32'h7FFF_FFFF};
    bins max_neg  = {32'h8000_0000};
    bins all_ones = {32'hFFFF_FFFF};
  }
```

Coverage: implementation example

Checking cross coverage:

```
cross_op_a: cross cp_op, cp_a_values {
    // Only care about add for this specific cross bin
    ignore_bins not_add = !binsobj(cp_op.ops[4'b0010]);
  }

endgroup
```

To run in actual testbench:

```
alu_coverage_cg alu_cg;
  function new(string name, uvm_component parent);
    super.new(name, parent);
    alu_cg = new(); // initialize the covergroup
  endfunction

// sample the coverage for this transaction
    alu_cg.sample(tr.alu_ctrl, tr.a, tr.b);
  endfunction
endclass
```

Test plans, regressions, coverage closure

- Usually specify features that must work beforehand and how it will be tested
  - Can include in modern EDA tools!
- Regression: automated process of running hundreds or thousands of randomized tests, makes sure changes are compatible
  - If I change block A, how do i know that doesnt accidentally break Block B?
  - Include a “smoke test”: resets, register access (R/W), send one normal simple transaction under default speeds and sizes
- Coverage closure: sign-off process where verification teams proves that the chip has been thoroughly tested and is ready to be manufactured

Formal Verification

Formal verification overview

Dynamic vs static verification

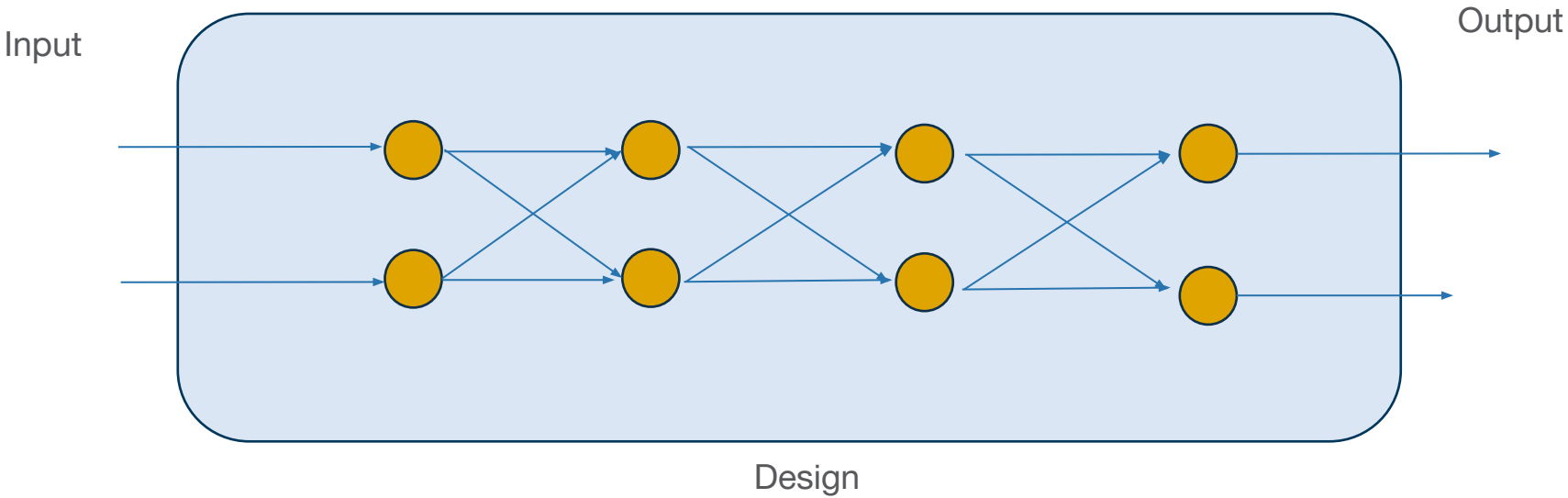
Formal verification is static

- Use mathematical induction and boolean algebra to evaluate the state space
- Provide constraints (what inputs are allowed) and properties (what the RTL must or must not do)

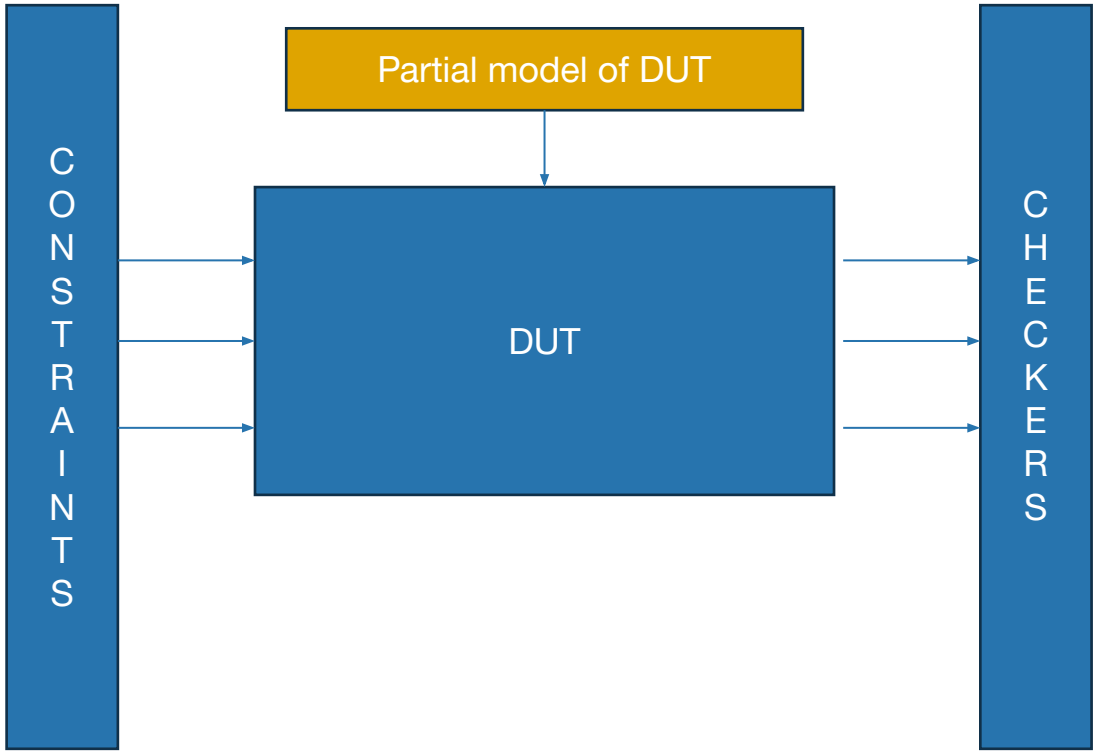
Types of proofs:

- Bounded proof: assertion cannot be violated within a specific number of clock cycles from reset -> no state space explosion
- Unbounded proof: no matter what sequence of inputs occurs and how long sim runs, the assertion can never be violated
- Falsification: provides a counter-example when property fails

Formal Verification and State Space

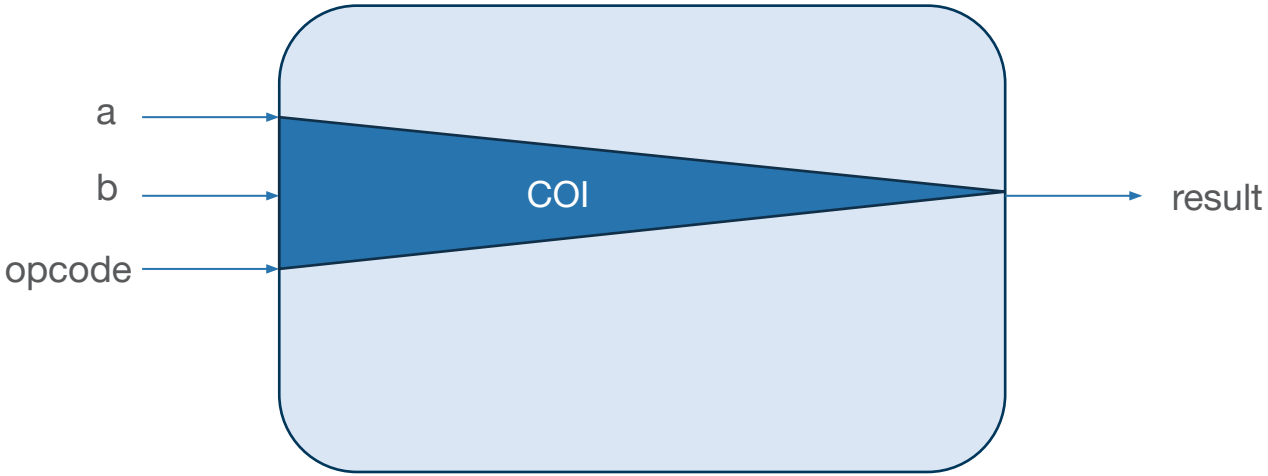


Formal Verification



Cone of Influence (COI)

Cone of influence: all the inputs/internal variables that influence a checker

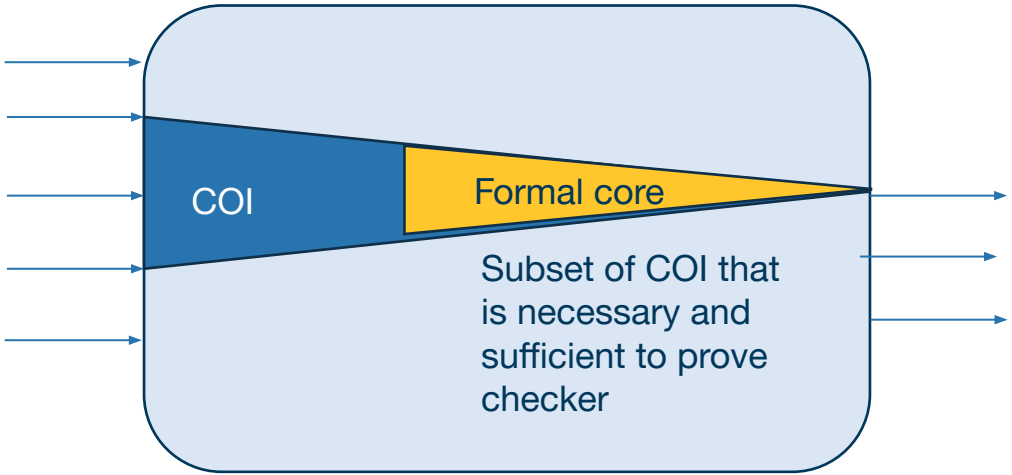



```
assert property (@(posedge clk)
    (alu_ctrl == 4'b0010) |-> (result == (a + b))
);
```

Cone of Influence (COI)

Process of determining COI:

1) Start at final equation, add all variables on RHS of RTL assignments to COI
2) For each variable just added, find assignment and add all of variables on RHS to COI
3) Continue until you reach inputs of the RTL design



Formal Verification: Constraints

Implemented by SVA assume statements:

Be careful of over constraining

- ALU example:

```
assume property (@(posedge clk)
    alu_ctrl inside {4'b0000, 4'b0001, 4'b0010, 4'b0110, 4'b0111} //opcode must be one of these
);
assume property (@(posedge clk) operand_a < 32'h0000_FFFF); //limits range of inputs
assume property (@(posedge clk) operand_b < 32'h0000_FFFF); /limits range of inputs
```

- Handshaking:

```
assume property (@(posedge clk)
    (valid_in && !ready_out) |-> ##1 $stable(operand_a) && $stable(operand_b)
);
```

Formal Verification: Checkers

Implemented by SVA assert statements:

- ALU example (functional correctness):

```
assert property (@(posedge clk)
    (alu_ctrl == 4'b0010) |-> (result == (a + b))
);
assert property (@(posedge clk)
    (result == 32'h0) <-> (zero_flag == 1'b1)
);
```

Handshaking:

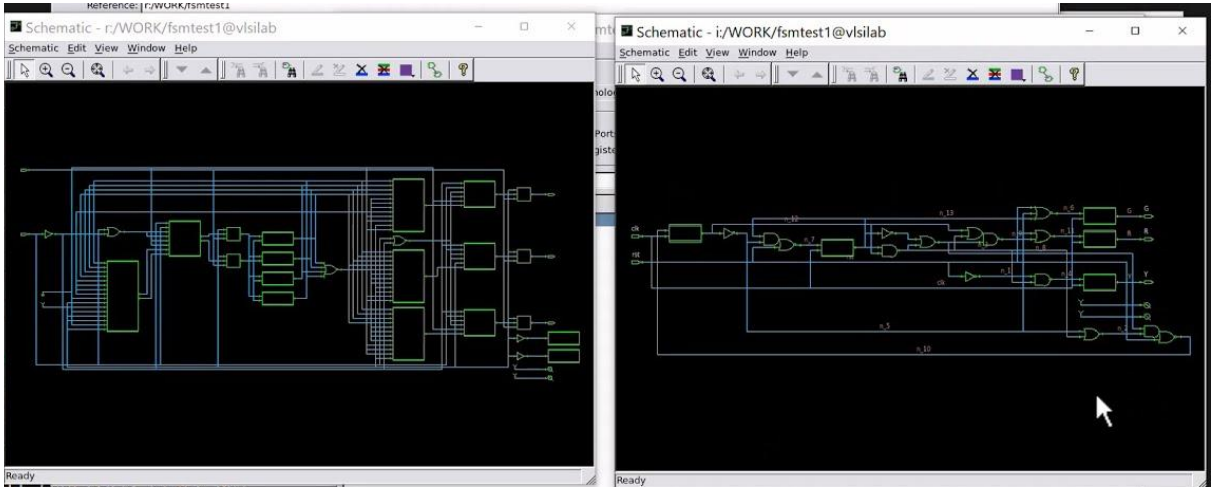
```
assert property (@(posedge clk)
    (valid_in && ready_out) |-> ##1 valid_out
);
```

If a case is found that breaks the assertion, stop immediately and provide waveform of inputs that cause error (counter example or cex)

Logic Equivalence Checking

Logic Equivalence Checking: type of formal verification where you compare two designs

- RTL against netlist (gate level representation generated by synthesis)
- Checks for X-propagation, pipeline and FSM equivalence of implemented and reference designs



Other Verification Methodologies

Methodology	Speed	Scale	Visibility	Use case
Formal	Very Fast	Block/IP level	Perfect	Checking important blocks like security or edge cases
Simulation	Slow (kHz)	Sub-system /small SoC	Good	Coverage closure + functional feature testing
Emulation	Fast (Mhz)	Full SoC / Multi-chip	Medium	Booting OS/production software, long stress-tests
FPGA Prototyping	Very fast (tens of Mhz)	Full SoC	Poor	Real-time interface testing, high speed hardware-software verification

Tools + Resources

Tools

Functional verification:

- Verdi/VCS (Synopsys)
- Incisive/Xcelium (Cadence)
- ModelSim/Questa

SVA cheatsheet:
https://uobdv.github.io/Design-Verification/Quick-References/SVA_QuickReference.CDNS.pdf

Comparing with non-sv/Verilog models:

Python: cocotb

C/C++: DPI-C (use through an import statement) or simulator APIs: VPI

Formal verification tools: JasperGold (Cadence), VC Formal (Synopsys), Yosys

LEC: Formality (Synopsys)

SPI Protocol

What is SPI

- SPI is a serial communication protocol
  - Full-duplex
  - Master controls the clock
  - Uses 4 signals: SCLK, MOSI, MISO, SS/SC
  - No built-in acknowledgement
  - One master to 1+ slaves

SPI Signals

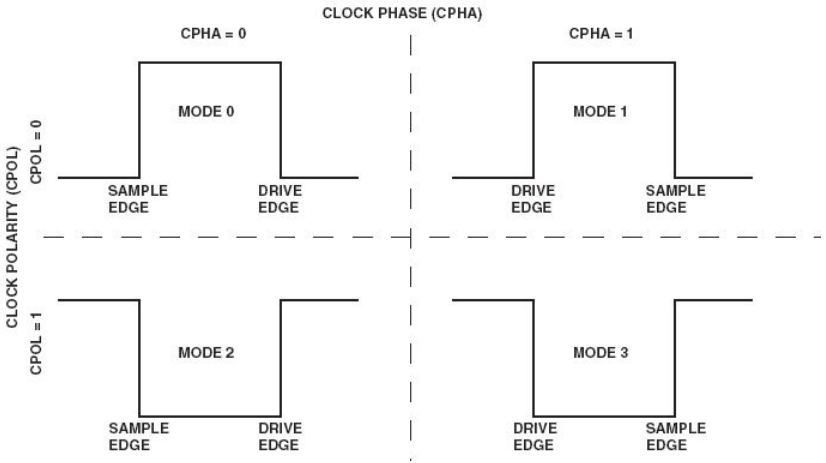
SCLK	Master → Slave	Serial clock
MOSI	Master → Slave	Master Out Slave In
MISO	Slave → Master	Master In Slave Out
SS/SC	Master → Slave	Active-low chip select

SPI Modes

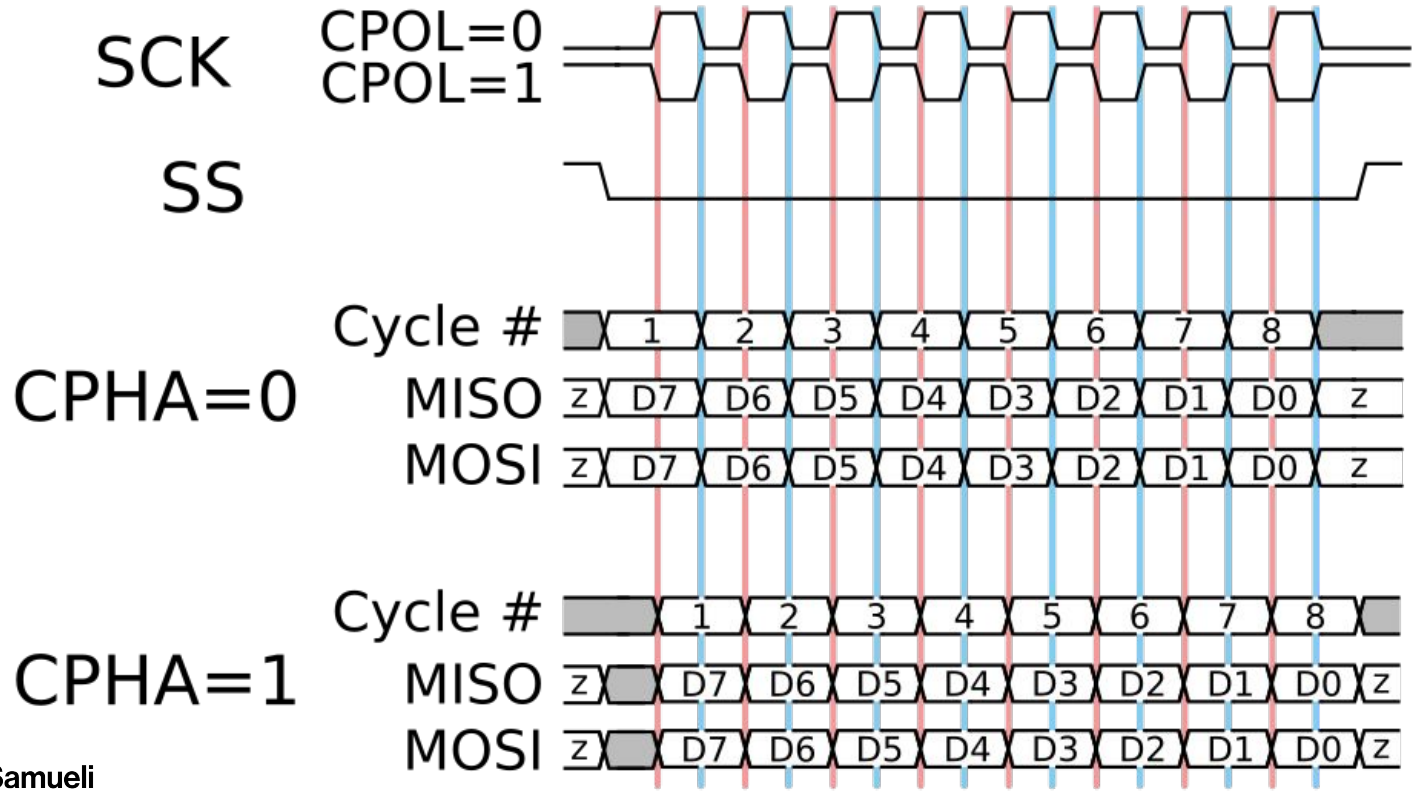
CPOL = idle level of SCLK

CPHA = data sampled on first or second edge

We will use mode 0



Example waveform



SPI Master

The master should:

- Generate SCLK from the system clock using a divider
- Assert and deassert CS_n
- Load a transmit shift register with tx_data
- Drive MOSI, MSB first
- Sample MISO into a receive shift register
- Count eight transmitted/received bits
- Assert done when the transfer completes.

Note: start should only be accepted when the master is idle, and done can be a one-cycle pulse

```
module spi_master (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       start,
    input  logic [7:0] tx_data,
    input  logic       miso,

    output logic       sclk,
    output logic       mosi,
    output logic       cs_n,
    output logic [7:0] rx_data,
    output logic       busy,
    output logic       done
);
```

SPI Slave

The slave should:

- Detect when CS_n goes low.
- Prepare its outgoing data on MISO.
- Sample MOSI on rising SCLK edges (Mode 0).
- Shift out MISO bits on falling SCLK edges.
- Count received bits.
- Latch the received byte and signal completion.
- Reset its transaction state when deselected.
- Sclk is treated like a signal → Synchronization happens **WITHIN** the SPI slave

Note: slave does NOT generate SCLK.

```
module spi_slave (
    input  logic       rst_n,
    input  logic       sclk,
    input  logic       cs_n,
    input  logic       mosi,
    input  logic [7:0] tx_data,

    output logic       miso,
    output logic [7:0] rx_data,
    output logic       done
);
```

Q&A